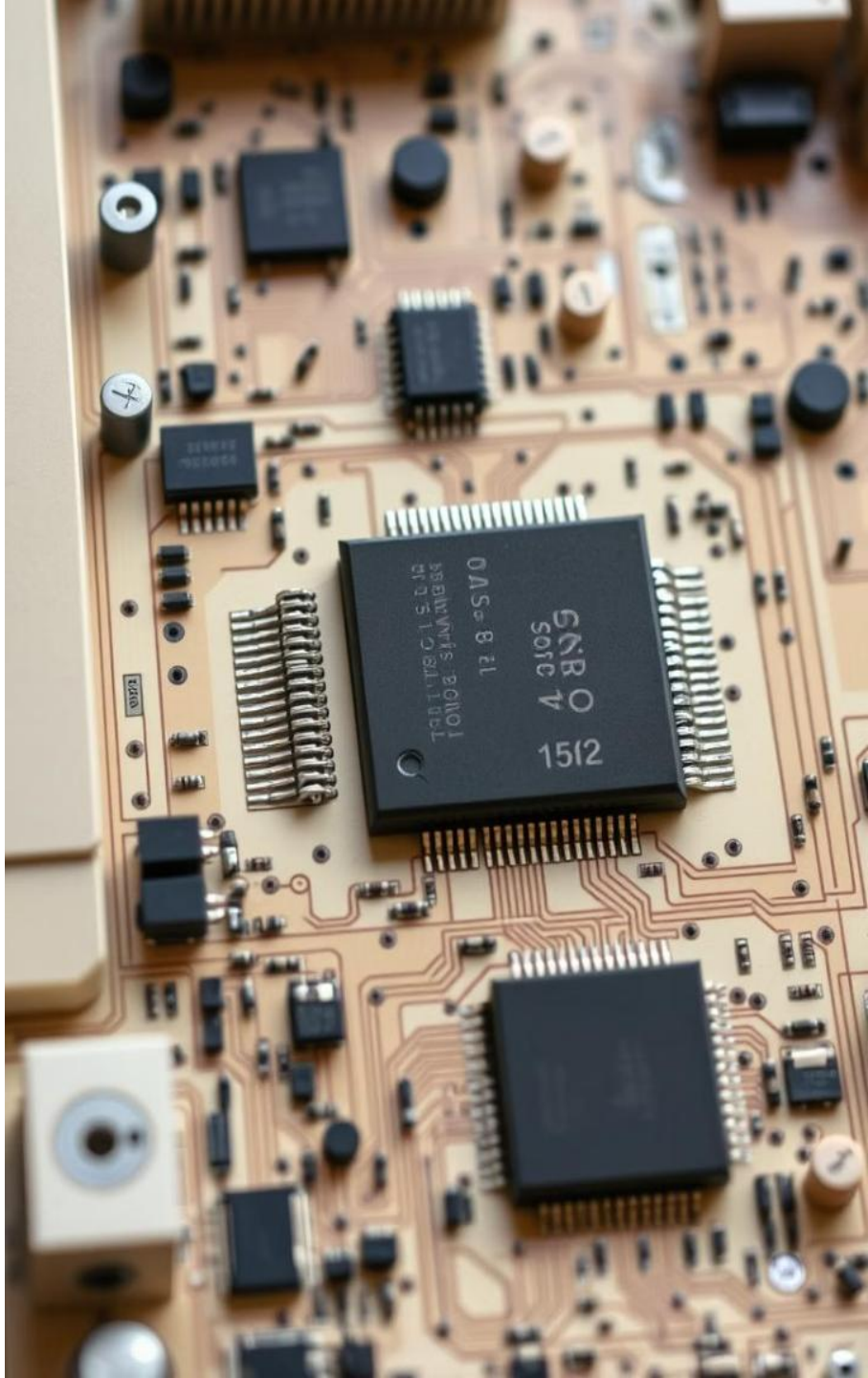
Course Title: **Parallel** and **Cloud**

Computing

Course Number: CABSXO5003

Introduction to OpenMP#1

Dr. Asif Irshad Khan
Dept. of Computer Science,
AMU, Aligarh



What is OpenMP?

OpenMP is a standard API (Application Programming Interface) used to write **shared–memory parallel** programs in C, C++, and Fortran.

It Consists of

Compiler Directives

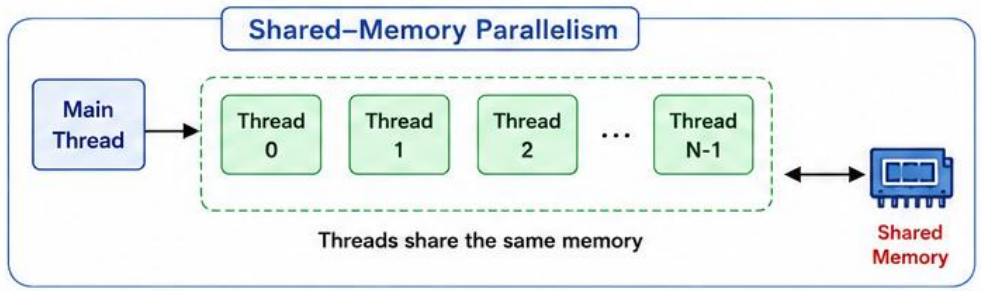
Special instructions (pragmas) added in the code that tell the compiler how to run code in parallel.

Runtime Routines

Library functions that provide information about threads, timers, and environment.

Environment Variables

External settings that control the behavior of the OpenMP runtime system.



Key Points

- OpenMP is a de-facto (widely adopted) standard for shared–memory programming.
- It works on multicore CPUs and other shared–memory architectures.
- Simple to use: add small directives to existing sequential code.
- Improves performance by executing tasks in parallel.

OpenMP Core Syntax

1. Compiler Directives

Most constructs in OpenMP are compiler directives.

General form:

```
#pragma omp construct [clause [clause] ...]
```

Example:

```
#pragma omp parallel num_threads(4)
```

2. Header File

Function prototypes and types are declared in:

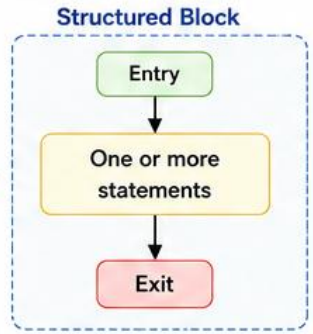
```
#include <omp.h>
```

3. Structured Block

Most OpenMP constructs apply to a “structured block”.

Example

```
#pragma omp parallel
{
   // statements
   if (condition)
      exit(0);  // allowed
   // more statements
}
```



- One entry point at the top.
- One exit point at the bottom.
- It is OK to have exit() inside the block.

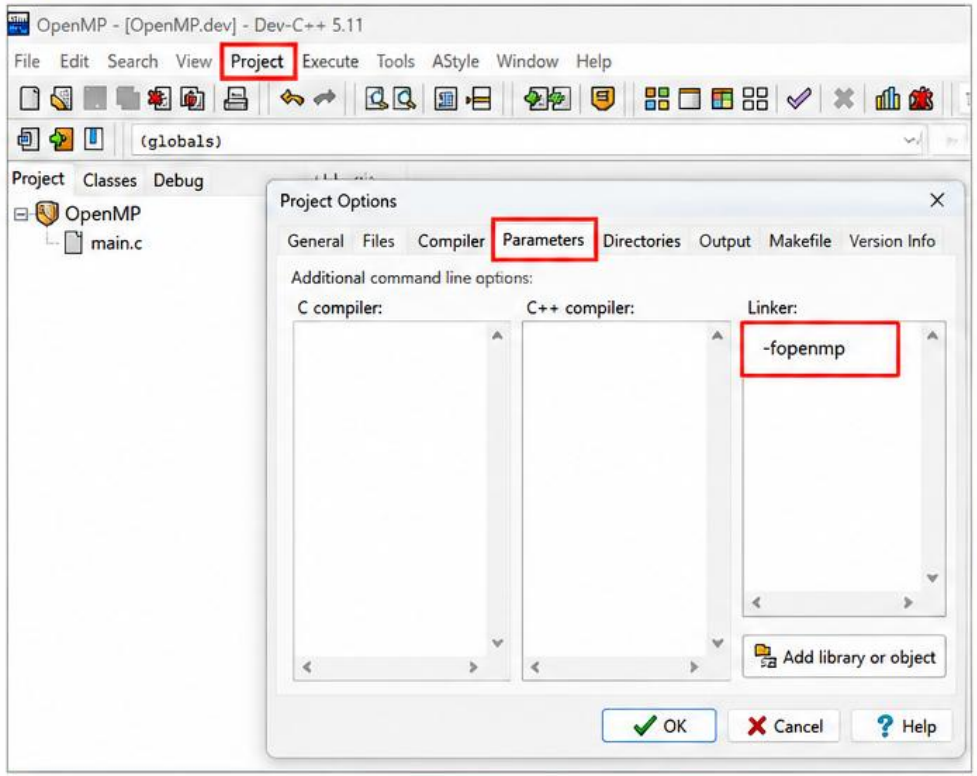
Key Points

- Use *#pragma omp ...* directives to define parallel regions and work.
- Include <omp.h> to access OpenMP library functions.
- OpenMP constructs work on structured blocks with single entry and exit.
- exit() is allowed inside the structured block.

OpenMP Project Setting in Dev-C++ Environment

To use OpenMP in Dev-C++, you must add the OpenMP compiler flag (**-fopenmp**) to both the compiler and the linker settings.

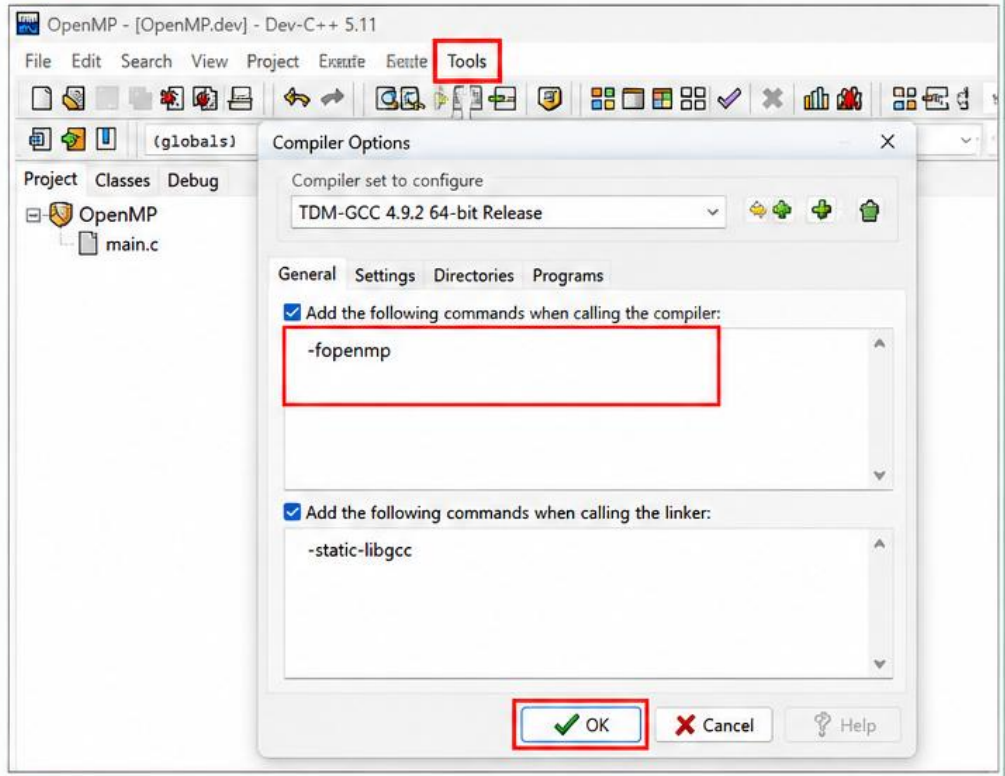
1 Go to Project → Project Options...



In the **Project Options** window:

- Go to the **Parameters** tab.
- In the **Linker** box, add: **-fopenmp**

2 Go to Tools → Compiler Options...



In the **Compiler Options** window:

- Under "Add the following commands when calling the compiler", add: **-fopenmp**
- Click **OK**.

3 Click OK in the Project Options window.

➡ **Now your Dev-C++ environment is ready for OpenMP programming!**

Exercise 1, Part A: Hello world

Goal: Verify that your environment works.

Task: Write a program that prints "hello world".

hello.c

```c
#include <stdio.h>

int main()
{
    int ID = 0;
    printf("hello(%d) ", ID);
    printf("world(%d) \n", ID);

    return 0;
}
```

What happens?

The program runs with a single thread. ID is 0.

Sample Output

```
hello(0) world(0)
```

Key Points

- This confirms your compiler and environment are set up correctly.

Exercise 1, Part B: Hello world (OpenMP)

Goal: Verify that your OpenMP environment works.

Task: Write a multithreaded program that prints "hello world".

hello_omp.c

```c
#include <stdio.h>
#include <omp.h>

int main()
{
   #pragma omp parallel
   {
      int ID = 0;
      printf("hello(%d) ", ID);
      printf("world(%d) \n", ID);
   }
   return 0;
}
```

Compilation (examples)

```
gcc   -fopenmp   hello_omp.c   -o hello_omp
gcc   -mp        hello_omp.c   -o hello_omp
pgcc  -mp        hello_omp.c   -o hello_omp
icx   /Qopenmp   hello_omp.c   -o hello_omp
```

What happens?

The program runs with multiple threads (default number depends on your system).

Sample Output (order may vary)

```
hello(0) world(0)  hello(2) world(2)
hello(1) world(1)  hello(3) world(3)
                ...
```

Key Points

- `#include <omp.h>` is required to use OpenMP.
- `#pragma omp parallel` creates a parallel region.
- Compilation requires OpenMP flags (examples shown above).

Exercise 1: Solution

A multi-threaded "Hello world" program

hello_omp.c

```c
#include <stdio.h>
#include <omp.h>

int main()
{
   #pragma omp parallel
   {
      int ID = omp_get_thread_num();
      printf("hello(%d) ", ID);
      printf("world(%d) \n", ID);
   }
   return 0;
}
```

- OpenMP include file
- Start of the parallel region
- Create a team of threads (default number)
- Get the ID of the calling thread
- End of the parallel region

How it works

1. `#pragma omp parallel` creates a parallel region.
2. All threads in the team execute the block inside { }.
3. `omp_get_thread_num()` returns a unique ID to each thread.
4. Each thread prints "hello world" with its own ID.

Sample Output (order may vary)

```
hello(0) world(0)   hello(2) world(2)   hello(3) world(3)
hello(1) world(1)   hello(4) world(4)   ...
```

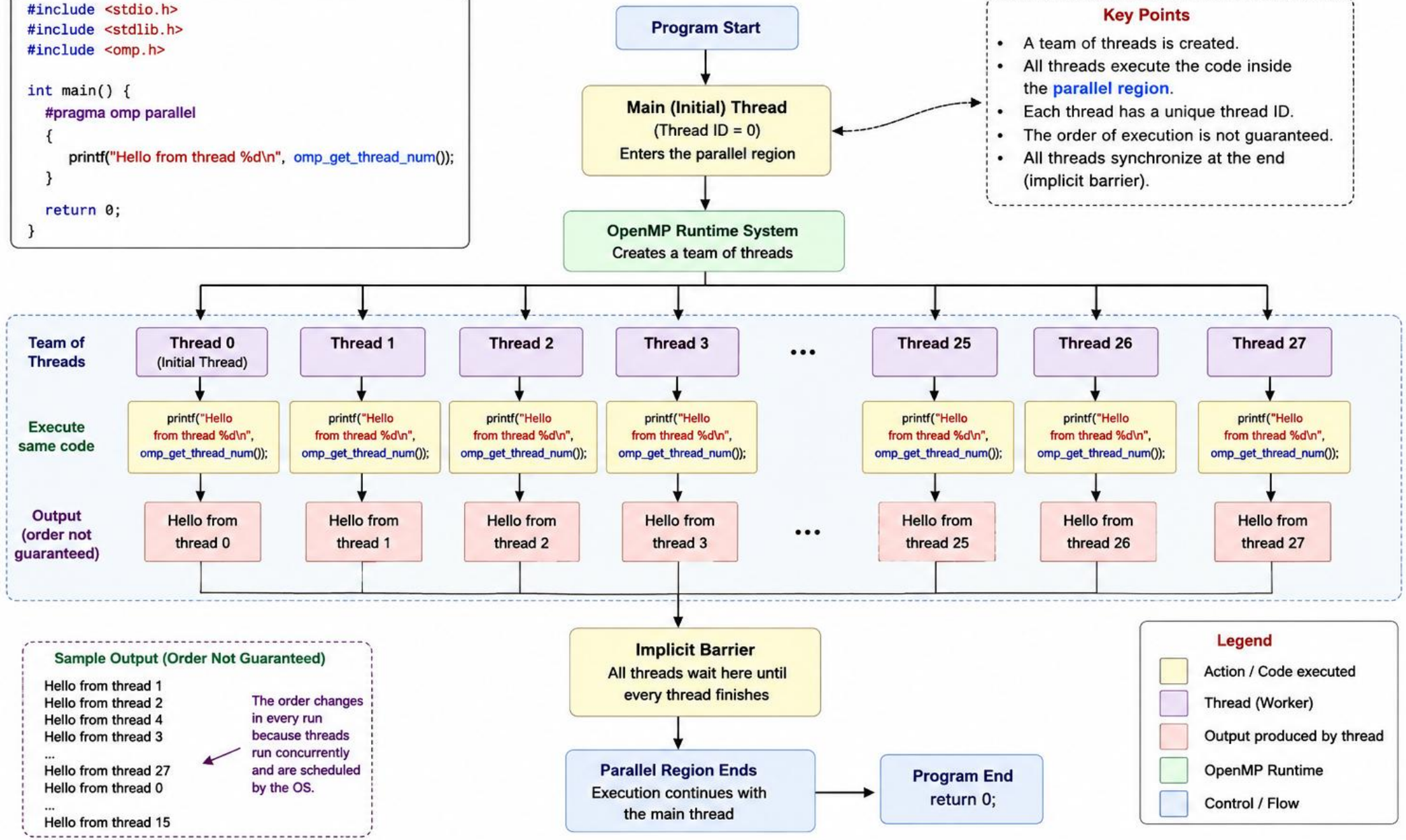
Key Takeaways

- OpenMP uses simple directives to enable parallelism.
- Parallel regions run with multiple threads concurrently.
- `omp_get_thread_num()` identifies each thread.
- Easy to start: add a few lines and compile with OpenMP flags!

What Happens Internally in OpenMP?

Execution of a Parallel Region

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
  #pragma omp parallel
  {
     printf("Hello from thread %d\n", omp_get_thread_num());
  }

  return 0;
}
```



Hello World! in Parallel (OpenMP)

C Program

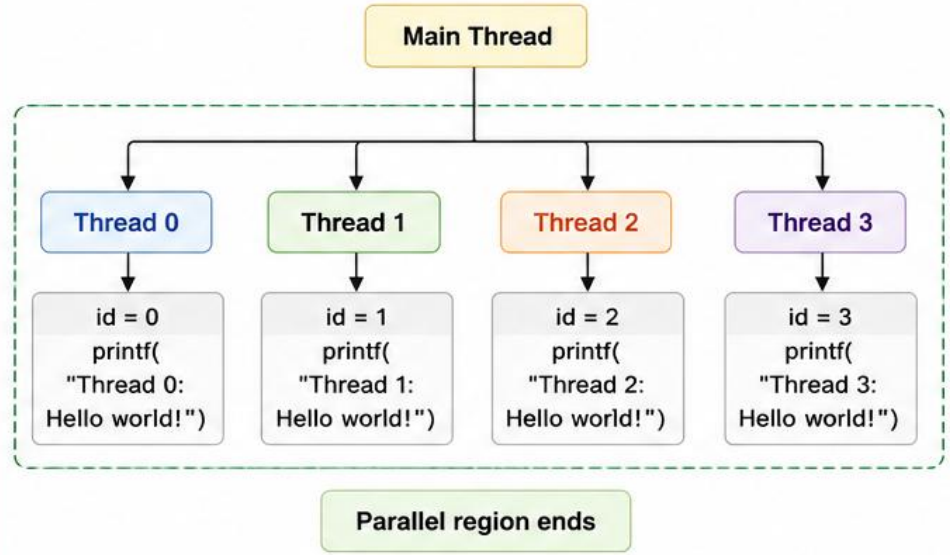
```c
#include <stdio.h>
#include <omp.h>

void main()
{
    omp_set_num_threads(4);   // Set number of threads to 4
   int id;
  #pragma omp parallel
   {
      id = omp_get_thread_num();   // Get thread ID
      printf("Thread %d: Hello world!\n", id);
   }
   return;
}
```

Total number of threads = 4

Each thread gets a unique thread ID (0 to 3)

What Happens?

OpenMP creates a team of 4 threads. Each thread executes the same block of code inside the parallel region.



Compile and Run

Compile (using GCC):

```
gcc -fopenmp hello_world.c -o hello_world
```

Run the program:

```
./hello_world
```

Sample Output (Order may vary)

```
Thread 0: Hello world!
Thread 1: Hello world!
Thread 2: Hello world!
Thread 3: Hello world!
```

OR

```
Thread 2: Hello world!
Thread 0: Hello world!
Thread 3: Hello world!
Thread 1: Hello world!
```

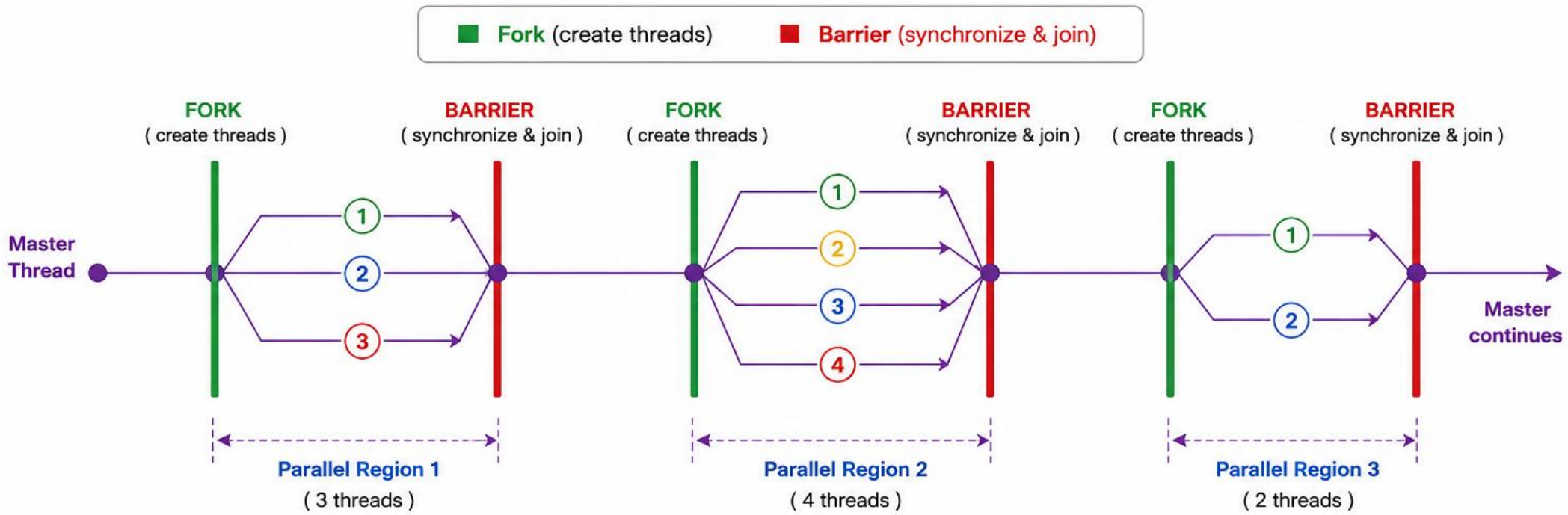
Note: The order of output may change from run to run.

Key Takeaways

- `omp_set_num_threads(n)` sets the number of threads to be used.
- `#pragma omp parallel` creates a team of threads that share the workload.
- `omp_get_thread_num()` returns the unique ID of the calling thread.
- All threads execute the same code inside the parallel region.
- Simple "Hello World" is the first step toward parallel programming!

Fork/Join Model in OpenMP

OpenMP programs start with a single **master thread**.
At each parallel region, the master thread **forks** a team of threads.
At the end of the region, all threads synchronize **(barrier)** and **join** back to the master thread.



Key Takeaways

FORK: At the start of a parallel region, the master creates a team of worker threads.

PARALLEL EXECUTION: All threads execute the statements in the parallel region concurrently.

BARRIER: At the end of the parallel region, all threads synchronize. Fast threads wait for the slowest one.

JOIN: After synchronization, all threads terminate (except the master), and execution continues with the master thread.

OpenMP Fork/Join Model

OpenMP follows the **fork/join model** for parallel execution.

Key Points

1 Start with a single thread

OpenMP programs start with a single **master thread**.

2 Fork: Create worker threads

At the start of a parallel region, the master thread creates a team of parallel "**worker**" threads. This is called **FORK**.

3 Execute in parallel

All threads (master + workers) execute the statements in the parallel region **concurrently**.

4 Join: Synchronize and merge

At the end of the parallel region, all threads synchronize and terminate, and control returns to the master thread. This is called **JOIN**.

Implicit barrier: There is an implicit barrier at the end of a parallel region.

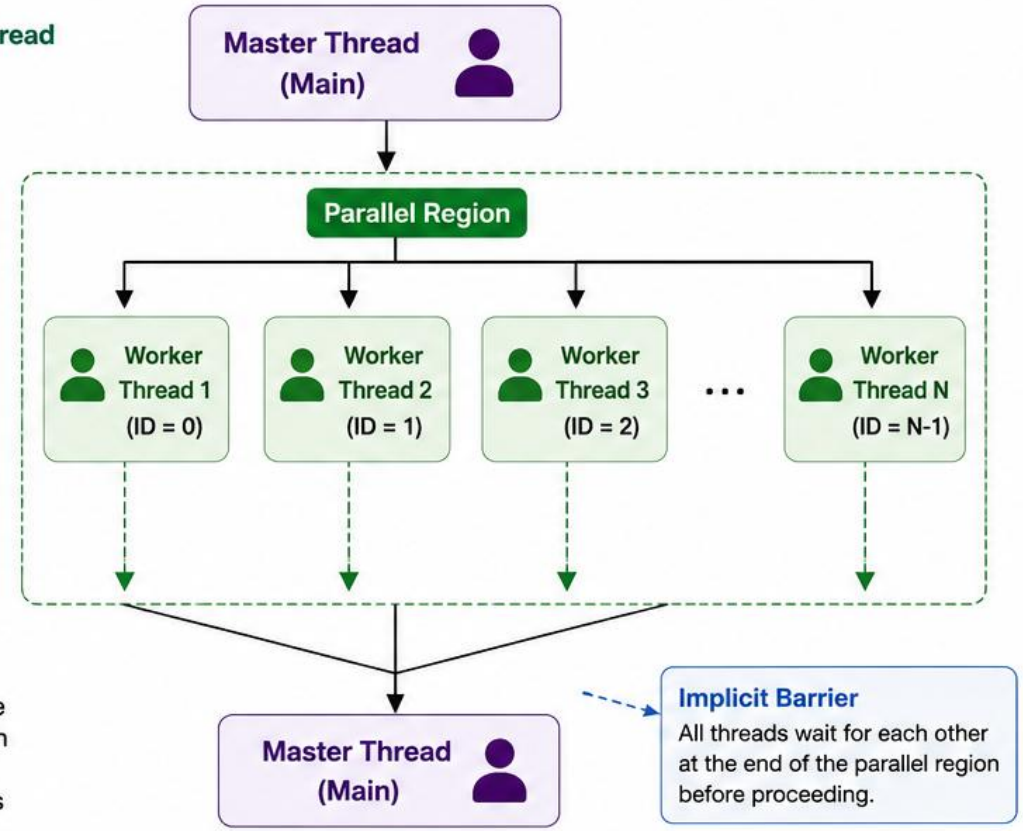
Visual Overview

1 Single Master Thread
Program starts.

2 FORK
Master creates a team of worker threads at the start of the parallel region.

3 Execute in Parallel
All threads execute **the statements in** the parallel region **concurrently**.

4 JOIN
All threads synchronize at the end of the region and **join the master** thread. Control returns to the **master.**



In Short

Start with one **master** thread. → **Fork:** Master creates multiple **worker** threads. → All threads execute the work in **parallel** inside the region. → **Join:** Threads synchronize and merge back.

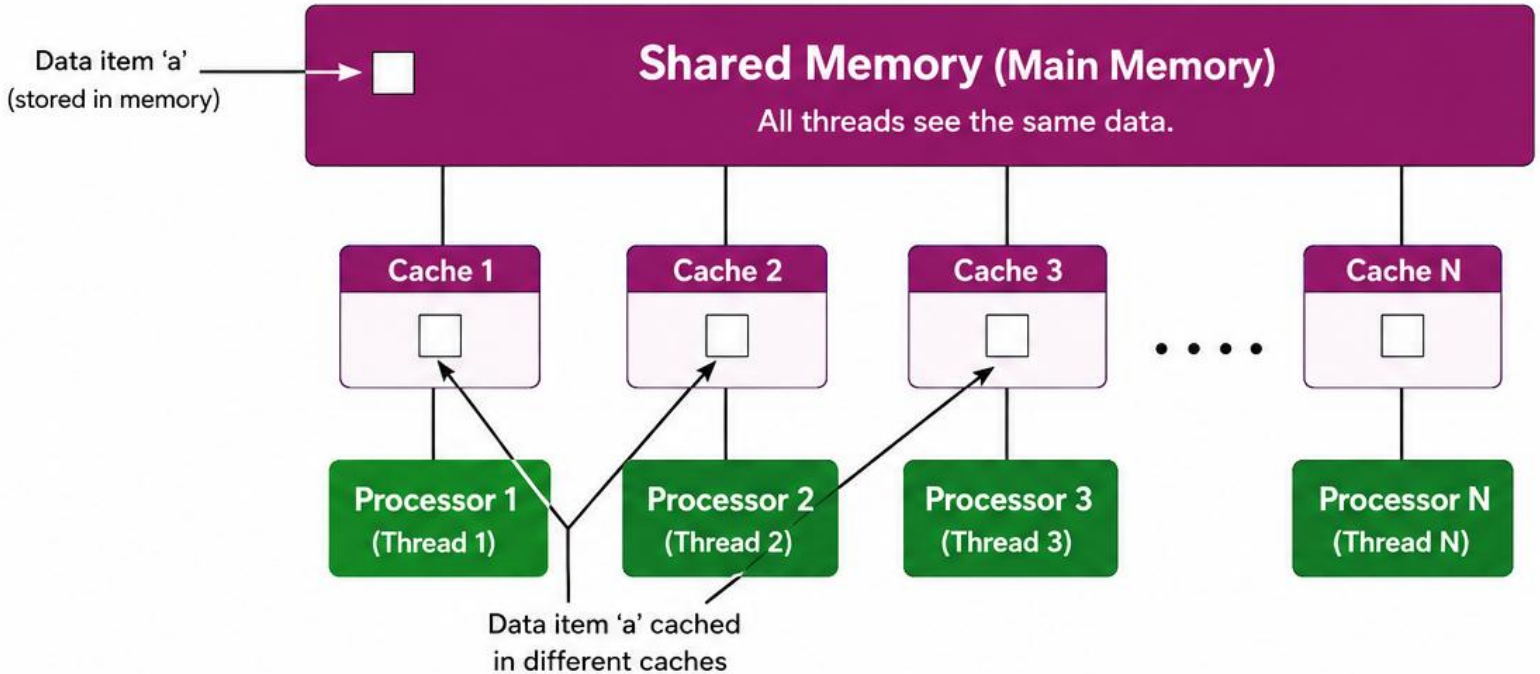
The **Fork/Join Model** is the basic execution model used by OpenMP to manage parallelism.

🔑 FORK vs JOIN

Stage	Meaning
Start	One master thread
FORK	Master creates a team of worker threads
Parallel Region	Threads execute work concurrently
Implicit Barrier	Threads wait for one another
JOIN	Worker threads finish and execution continues

OpenMP Memory Model

- OpenMP uses a **shared memory model**.
- All threads **share the same address space**, but due to hardware, it can get complicated.



How it works

1. A variable (e.g., 'a') is stored in shared memory.
2. Each processor (core) may keep its own copy of the data in its local cache.
3. All threads access the same memory location, but may read/write through their own cache.
4. Caches help performance but can cause complexity (e.g., stale data, consistency).

Key Takeaways

- OpenMP is a **shared memory** parallel programming model.
- All threads share the **same address space** and global memory.
- **Multiple copies** of data may exist in main memory, caches, or registers.
- **Caches** improve performance but may introduce complexity.
- Understanding the memory hierarchy is important for **correctness and performance** (synchronization, consistency).

In short: Threads share the same memory, but hardware (caches, registers) keeps multiple copies of data, making memory behavior more complex.

OpenMP Threads vs Cores

What are threads, cores, and how do they relate?

Key Concepts

Thread
An independent sequence of execution of program code.
- A block of code with one entry and one exit.

Core (CPU Core)
A physical processing unit in a CPU.
- Cores execute threads.

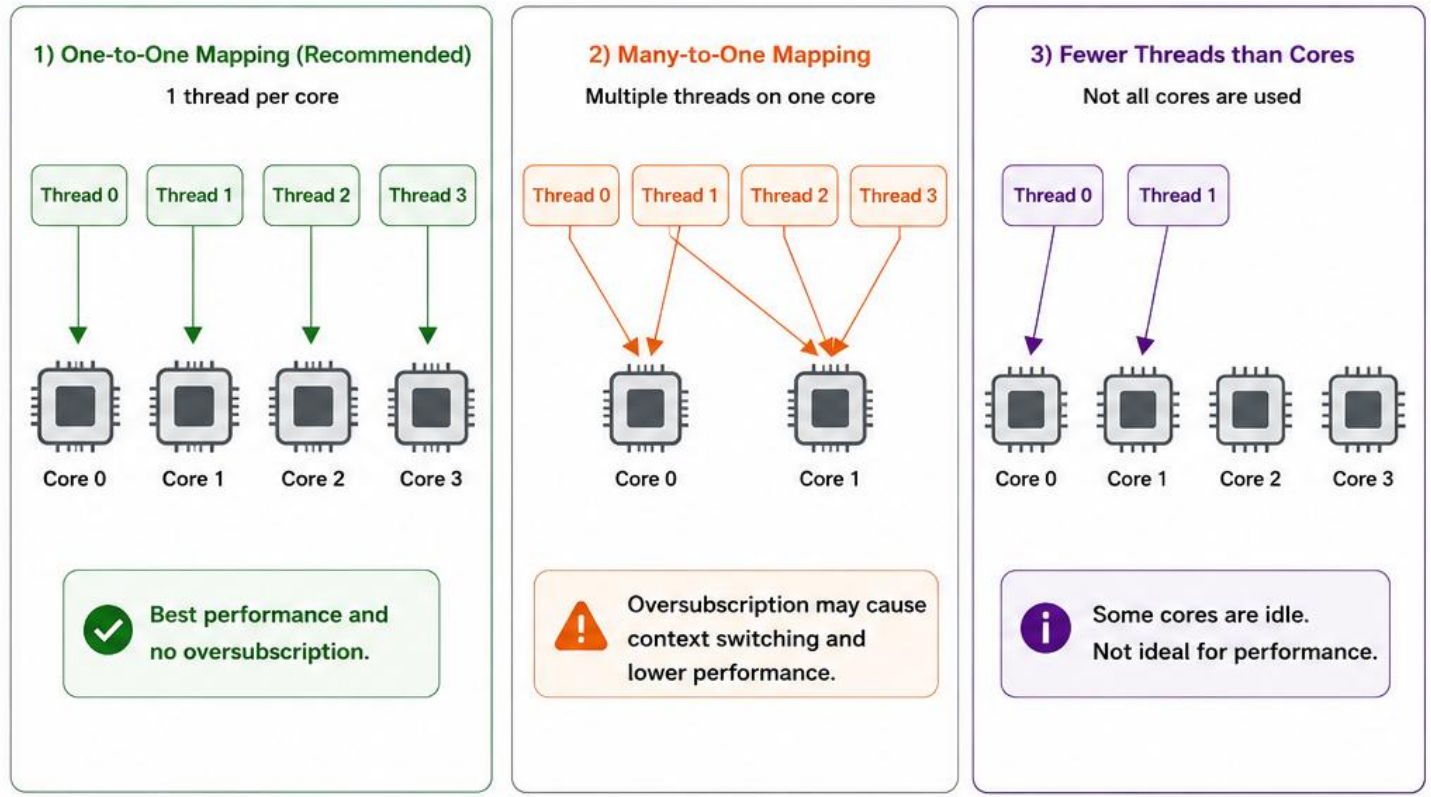
Abstraction
Threads are a more abstract concept than cores/CPUs.

Mapping
OpenMP threads are mapped onto physical CPU cores by the system.

Flexibility
It is possible to map more than one thread on a single core.

Best Practice
In practice, best performance is usually achieved with one-to-one mapping (1 thread per core).

How OpenMP Threads Map to CPU Cores



Key Takeaways

- A thread is a sequence of instructions.
- A core is a physical execution unit.
- Threads are abstract; cores are physical.
- More than one thread can run on a core, but it may reduce efficiency.
- Aim for one-to-one mapping for best performance.

More About OpenMP Threads

Setting the Number of OpenMP Threads

1) Using Environment Variable

Set the environment variable **OMP_NUM_THREADS** before running the program.

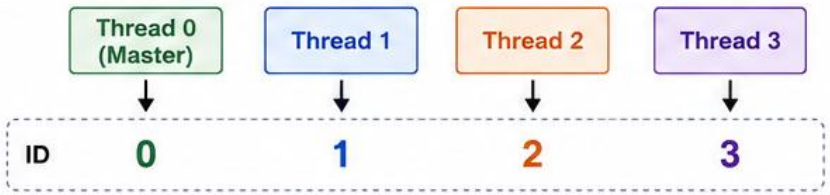
```
$ export OMP_NUM_THREADS=4   # Linux / macOS
$ ./a.out
```

2) Using Runtime Function

Set the number of threads in your program before entering the parallel region.

```
#include <omp.h>
...
omp_set_num_threads(n);  // set number of threads
```

Example: 4 Threads in a Parallel Region



Thread IDs are unique within a team and help threads identify their work.

Useful OpenMP Runtime Functions

omp_get_num_threads()

- **Returns** the number of threads in the current parallel region.
- If called outside a parallel region, returns 1.

```
int nt = omp_get_num_threads();
printf("Number of threads = %d\n", nt);
```

omp_get_thread_num()

- Returns the ID of the calling thread in the team.
- Value is between [0, n-1], where n = number of threads.
- The master thread always has ID 0.

```
int tid = omp_get_thread_num();
printf("Thread ID = %d\n", tid);
```

Summary

Function / Variable	Purpose
OMP_NUM_THREADS	Environment variable to set number of threads.
omp_set_num_threads(n)	Runtime function to set number of threads.
omp_get_num_threads()	Get number of threads in current parallel region (or 1 outside).
omp_get_thread_num()	Get ID of calling thread in the team [0, n-1]; master has ID 0.

Key Takeaways

You can control the number of OpenMP threads using an environment variable or a runtime function.

omp_get_num_threads() tells you how many threads are in the current team.

omp_get_thread_num() gives each thread a unique ID from 0 to n-1 (master is 0).

These functions are essential for writing efficient and flexible parallel programs.

Example#

/* Your program is a simple example of using OpenMP to create a parallel region in a C program. In this code, each thread will print a message indicating its thread number. */

```
#include <stdio.h>
#include <stdlib.h>
//Include the OpenMP Header:
#include <omp.h>

int main(int argc, char *argv[])
{
   // Parallel region
   #pragma omp parallel
   {
      printf("Hello from thread %d\n", omp_get_thread_num());
   }

   return 0;
}
```

OpenMP Example: Print Thread Numbers

Example Code

```
1  /* Your program is a simple example of using OpenMP to create
2    a parallel region in a C program. In this code, each thread will
3    print a message indicating its thread number. */
4  #include <stdio.h>
5  #include <stdlib.h>
6  //Include the OpenMP Header:
7  #include <omp.h>
8
9  int main(int argc, char *argv[])
10 {
11    // Parallel region
12    #pragma omp parallel
13    {
14        printf("Hello from thread %d\n", omp_get_thread_num());
15    }
16   return 0;
17 }
```

Required header files for standard functions and OpenMP.

Program starts execution from here.

Starts a parallel region: all threads execute the block inside.

Returns the unique thread ID of the calling thread.

Breakdown of the Code

1. **Headers**
   The program includes necessary headers: `stdio.h` for standard input/output functions, `stdlib.h` for general utilities, and `omp.h` for OpenMP-specific functions.

2. **main Function**
   The program's entry point is the main function, which also accepts command-line arguments.

3. **Parallel Region**
   The `#pragma omp parallel` directive creates a parallel region. The code within this block will be executed by multiple threads concurrently.

4. **Thread Identification**
   `omp_get_thread_num()` returns the unique thread number (ID) of each thread. In the printf statement, each thread prints a message indicating its thread number.

What it does?

Creates a team of threads.
Each thread prints a message showing its own thread ID.

Sample Output (Order may vary)

```
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3
...
(Up to N threads depending on your system)
```

Order is not guaranteed because threads run concurrently.

Key Takeaways

- `#pragma omp parallel` creates a team of threads.
- Each thread executes the code inside the parallel region.
- `omp_get_thread_num()` helps identify each thread.
- Number of threads = set by environment or `omp_set_num_threads()`.
- Output order is not deterministic (may change every run).

Example#

/*Your C program utilizes OpenMP to perform parallel addition of elements from two arrays (a and b) and stores the results in a third array (c). Each thread is responsible for adding corresponding elements from the a and b arrays and printing the result.*/

```
#include<stdio.h>
#include<omp.h>

int main(int argc, char *argv[])
{
        int a[5]={1,2,3,4,5};
        int b[5]={6,7,8,9,10};
        int c[5];
        int tid;

        #pragma omp parallel num_threads(5)
        {
                tid=omp_get_thread_num(); /* get the thread id*/
                c[tid]=a[tid]+b[tid];  /*each operation on individual core*/
                printf("c[%d]=%d\n",tid,c[tid]);
        }

}
```

OpenMP Example: Parallel Addition of Two Arrays

This C program uses OpenMP to perform parallel addition of elements from two arrays (a and b) and stores the results in a third array (c). Each thread is responsible for adding corresponding elements from the arrays and printing the result.

C Program

```c
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[])
{
    int a[5] = {1, 2, 3, 4, 5};
    int b[5] = {6, 7, 8, 9, 10};
    int c[5];
    int tid;

 #pragma omp parallel num_threads(5)
 {
    tid = omp_get_thread_num();     // get the thread id
    c[tid] = a[tid] + b[tid];       // add on individual core
    printf("c[%d] = %d\n", tid, c[tid]);
 }
}
```

Arrays and variables are declared.

Creates a parallel region with 5 threads.

Each thread works on its own index (tid).

Each thread prints its part of the result.

Breakdown of the Code

1. **Array Initialization**
   Two arrays, a and b, are initialized with values:
   `a = {1, 2, 3, 4, 5}` `b = {6, 7, 8, 9, 10}`
   An array c is declared to store the result of adding elements from a and b.

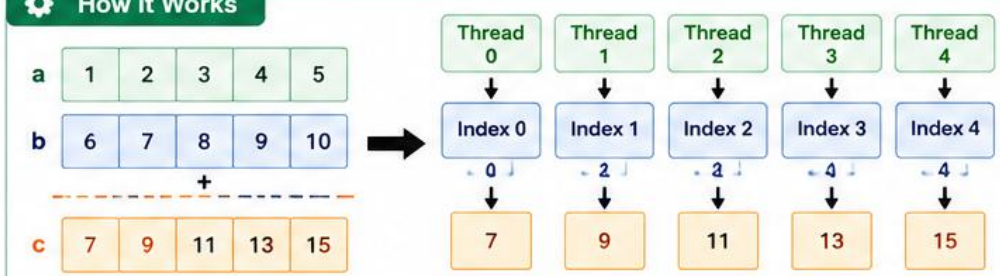
2. **Parallel Region**
   The `#pragma omp parallel num_threads(5)` directive creates a parallel region with 5 threads.

3. **Thread ID**
   `tid = omp_get_thread_num()` gets the unique thread ID for each thread (ranging from 0 to 4).

4. **Array Addition**
   Each thread performs addition on elements of a and b arrays using its unique thread ID as an index.

5. **Output**
   Each thread prints its part of the result.

How it Works



Each thread handles one index and computes c[index] = a[index] + b[index]

Sample Output (Order may vary)

```
c[0] = 7
c[1] = 9
c[2] = 11
c[3] = 13
c[4] = 15
```

(Order of lines may change in every run)

Key Takeaways

- OpenMP makes it easy to run code in parallel.
- `#pragma omp parallel` creates a team of threads.
- `omp_get_thread_num()` gives each thread a unique ID.
- Each thread works independently on its own data.
- Results may be printed in any order.
- Useful for data-parallel tasks like array operations.

Example#

/*Write a program that multiplies each element of an array by 2 using multiple threads.*/

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[]) {
   int array[5] = {1, 2, 3, 4, 5};  // Array of numbers
   int i;  // Loop counter

   // Parallel region with simple for loop
   #pragma omp parallel for
   for (i = 0; i < 5; i++) {
      array[i] *= 2;  // Multiply each element by 2
      printf("Thread %d: array[%d] = %d\n", omp_get_thread_num(), i, array[i]);
   }

   // Print the final array after modification
   printf("Final array values:\n");
   for (i = 0; i < 5; i++) {
      printf("%d ", array[i]);
   }
   printf("\n");

   return 0;
}
```

Example

Write a program that multiplies each element of an array by 2 using multiple threads.

C Program

```c
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    int array[5] = {1, 2, 3, 4, 5};   // Array of numbers
    int i;   // Loop counter

    // Parallel region with simple for loop
    #pragma omp parallel for
    for (i = 0; i < 5; i++) {
        array[i] *= 2;   // Multiply each element by 2
        printf("Thread %d: array[%d] = %d\n",
                omp_get_thread_num(), i, array[i]);
    }

    // Print the final array after modification
    printf("\nFinal array values:\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", array[i]);
    }
    printf("\n");

    return 0;
}
```

Sample Output (with 4 threads)

```
Thread 0: array[0] = 2
Thread 2: array[2] = 6
Thread 1: array[1] = 4
Thread 3: array[3] = 8
Thread 0: array[4] = 10

Final array values:
 2  4  6  8  10
```

Note:
The order of thread prints may vary because threads execute concurrently.

Here's a breakdown of the code:

1. **Array Initialization**
   The array is initialized with the values `{1, 2, 3, 4, 5}`.

2. **Parallel Region**
   The `#pragma omp parallel for` directive is used to indicate that the following `for` loop should be executed in parallel. Each iteration of the loop is handled by a different thread.

3. **Multiplication Operation**
   Each thread multiplies its assigned element of the array by 2.

4. **Printing Thread Information**
   Inside the loop, each thread prints its ID (using `omp_get_thread_num()`) and the modified value of the array element it processed.

5. **Final Array Output**
   After the parallel region, the main thread prints the final values of the array.

Key Takeaways

- `#pragma omp parallel for` parallelizes a loop, distributing iterations among threads.
- `omp_get_thread_num()` gives the ID of the calling thread (from 0 to n-1).
- Threads execute concurrently, so the print order may vary.
- After the parallel region, all threads synchronize, and execution continues with the main thread.

Example#

/*Write a program Calculating the Cube of Array Elements Using OpenMP.*/

```
#include <stdio.h>
#include <omp.h>  // Include the OpenMP header

int main() {
    int n = 6;
    int a[] = {1, 2, 3, 4, 5, 6};  // Input array
    int c[n];  // Array to store the cubes

    // Parallel region to compute cubes
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        c[i] = a[i] * a[i] * a[i];
    }

    // Print results
    printf("Cubed elements:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", c[i]);
    }
    printf("\n");

    return 0;
}
```

Example

Write a program Calculating the Cube of Array Elements Using OpenMP.

C Program

```c
#include <stdio.h>
#include <omp.h>   // Include the OpenMP header

int main() {
    int n = 6;
    int a[] = {1, 2, 3, 4, 5, 6};    // Input array
    int c[n];    // Array to store the cubes

    // Parallel region to compute cubes
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        c[i] = a[i] * a[i] * a[i];   // Calculate cube
    }

    // Print results
    printf("Cubed elements:\n");
    for (int i = 0; i < n; i++) {
       printf("%d ", c[i]);
    }
    printf("\n");

    return 0;
}
```

Sample Output

```
Cubed elements:
1 8 27 64 125 216
```

Here's a breakdown of the code:

1 Include OpenMP Header

`#include <omp.h>` is included to use OpenMP functions.

2 Array Initialization

The `a[]` array is initialized with the values `{1, 2, 3, 4, 5, 6}`. The array `c[]` is declared to store the cube of each element.

3 Parallel Region

The `#pragma omp parallel for` directive is used to parallelize the for loop. This directive splits the loop iterations among the available threads.

4 Calculating Cubes

Each thread calculates the cube of the element assigned to it and stores it in the corresponding position in the `c[]` array.

5 Printing Results

After the parallel computation, the results are printed to show the cubes of the original array elements.

Key Takeaways

- `#pragma omp parallel for` distributes loop iterations among threads.
- Each thread works on a portion of the data independently.
- No explicit synchronization is needed here because each thread writes to a different index of `c[]`.
- This leads to faster execution on multi-core processors.

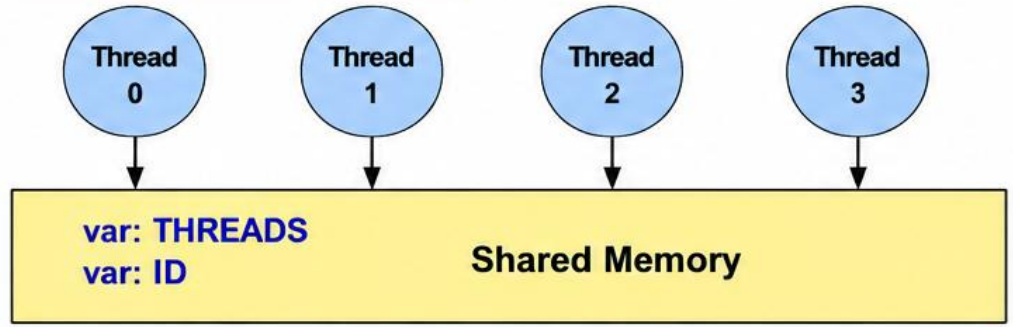
Shared Memory Model

The memory is (logically) shared by all the CPU's

Relevant piece of code

```
THREADS = omp_get_num_threads()
ID = omp_get_thread_num()
```

From our example on previous page



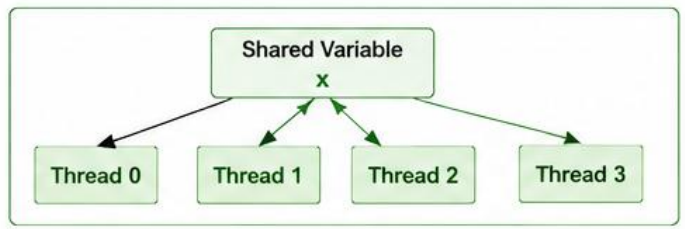
All threads try to access the same variable (possibly at the same time). This can lead to a race condition. Different runs of the same program might give different results because of race conditions.

Shared and Private Variables

OpenMP provides a way to declare variables private or shared within an OpenMP block.
This is done using the following OpenMP *clauses*:

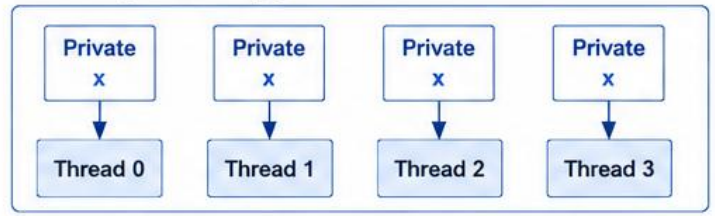
SHARED (*list*)

- All variables in *list* will be considered shared.
- **Every OpenMP thread has** access to all these variables.



PRIVATE (*list*)

- Every OpenMP thread will have its own "private" copy of variables in *list*.
- No other OpenMP thread has access to this "private" copy.



For example: `!$OMP PARALLEL PRIVATE(a,b,c)` (Fortran) or `#pragma omp parallel private(a,b,c)` (C/C++)

// Hello program with race condtion

```
#include <stdio.h>
#include <stdlib.h>
#include<omp.h>

int main(int argc, char *argv[]) {
        int numt, tid;
        #pragma omp parallel
        {
        numt= omp_get_num_threads();
        tid=omp_get_thread_num();
        printf(" hello from thread %d of %d\n",tid,numt);
  }
        return 0;
}
```

// Almost Failing Hello program

```
#include <stdio.h>
#include <stdlib.h>
#include<omp.h>

int main(int argc, char *argv[]) {
            int numt,tid;
            int j;
            #pragma omp parallel
            {
            numt= omp_get_num_threads();
            tid=omp_get_thread_num();
            for( j=0;j<=1000000;j++);
            printf(" hello from thread %d of %d\n",tid,numt);
  }
            return 0;
}
```

OpenMP Program (Race Condition)

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char *argv[]) {
   int numt, tid;
   #pragma omp parallel
   {
       numt = omp_get_num_threads();
       tid = omp_get_thread_num();
       printf("hello from thread %d of %d\n",tid,numt);
   }
   return 0;
}
```

Why There is a Race Condition?

`numt` and `tid` are shared variables by default.

▶ **Why?**

- In OpenMP, variables declared outside the parallel region are shared by all threads.

```c
/* Shared by all threads */
int numt, tid;
```

- Inside the parallel region:

```c
numt = omp_get_num_threads();
tid = omp_get_thread_num();
printf("hello from thread %d of %d\n", tid, numt);
```

- All threads are simultaneously doing:

```
Thread 0 → writes numt, writes tid → reads them
Thread 1 → writes numt, writes tid → reads them
Thread 2 → writes numt, writes tid → reads them
...
```

- So particularly `tid` is a problem.

Example of What Can Happen

Suppose there are 4 threads.

▶ Thread 0 might execute:

```
tid = 0
```

▶ but before it reaches `printf`, Thread 1 may execute:

```
tid = 1
```

▶ Then Thread 0 could print:

```
hello from thread 1 of 4
```

▶ instead of:

```
hello from thread 0 of 4
```

⚠ The exact output is nondeterministic.

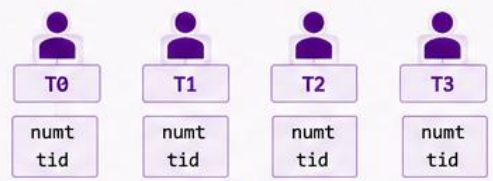
Correct Version (Avoid Race Condition)

Make `numt` and `tid` private to each thread:

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char *argv[]) {
   int numt, tid;
   #pragma omp parallel private(numt, tid)
   {
       numt = omp_get_num_threads();
       tid = omp_get_thread_num();
       printf("hello from thread %d of %d\n", tid, numt);
   }
   return 0;
}
```

What `private(numt, tid)` does?

- Each thread gets its own private copy of numt and tid.
- No other thread can modify those copies.
- Each thread prints the correct values.



Key Takeaways

- Variables declared outside the parallel region are shared by default.
- Modifying shared variables without synchronization can lead to race conditions.
- Use `private` clause (or other data scoping clauses) to give each thread its own copy.
- This ensures correct and deterministic behavior.

// fixing Hello program

```
#include <stdio.h>
#include <stdlib.h>
#include<omp.h>

int main(int argc, char *argv[]) {
        int numt,tid;
        #pragma omp parallel private ( tid ) shared( numt )
        {
        numt= omp_get_num_threads();
        tid=omp_get_thread_num();
        printf(" hello from thread %d of %d\n",tid,numt);
  }
        return 0;
}
```

Important Distinction

There are actually **two issues** worth teaching here:

Variable	Original Program	Correct Program	Explanation
`numt`	**Shared**	**Private**	Each thread should have its own copy of `numt`.
`tid`	**Shared**	**Private**	Each thread should have its own copy of `tid`.
`printf()`	**Multiple threads can execute it**	**Multiple threads can execute it**	Having multiple threads execute `printf()` is fine.
Race condition	**Yes**	**No**	Race condition occurs in the original program due to shared variables.

Note: However, `printf()` itself is generally protected by the C library/runtime sufficiently to avoid corrupting the individual output calls, but the **order of lines is not guaranteed**.

Best Teaching Point

The key rule for your students is:

> Variables declared **outside** an OpenMP parallel region are **shared** by default.
> Variables declared **inside** the parallel region are **private** to each thread.

OpenMP Constructs

OpenMP provides a set of simple but powerful constructs (directives) to express parallelism in C, C++, and Fortran programs.

1 Parallel Region

- Creates a team of threads to execute a block of code in parallel.
- Implicit barrier at the end (unless specified otherwise).

```
#pragma omp parallel
{
    // Code executed by all threads
}
```

2 Data Environment

- Controls how variables are shared among threads.
- Variables can be declared as shared or private.

```
#pragma omp parallel shared(a, b) private(c)
{
    // Code
}
```

3 Worksharing

- Distributes work among threads.
- Each thread performs a portion of the iterations.

```
#pragma omp for          // Distribute loop iterations
#pragma omp sections     // Distribute code sections
```

4 Synchronization

- Ensures correct coordination among threads.
- Helps prevent race conditions and maintain data consistency.

```
#pragma omp barrier      // Wait for all threads
#pragma omp critical     // Protect critical section
```

At a Glance

1 Parallel Region
Create threads and run in parallel

2 Data Environment
Define variable sharing behavior

3 Worksharing
Distribute work among threads

4 Synchronization
Coordinate threads and ensure correctness

Control Number of Threads

OpenMP provides three ways to set the number of threads.

1 

Parallel Region Clause

Specify the number of threads directly in the parallel region.

```
#pragma omp parallel num_threads(integer)
```

2 

Run-time Function

Set the number of threads at runtime using a function.

```
omp_set_num_threads(integer)
```

3 

Environment Variable

Set the environment variable before running the program.

```
OMP_NUM_THREADS
```

Priority

Highest Priority
(Parallel region clause)

Medium Priority
(Run-time function)

Lowest Priority
(Environment variable)

Note: If more than one method is used, the one with the highest priority takes precedence.